Experiment No.06

Aim: Apply K-Means and Hierarchical Clustering on sample datasets.

Theory:

1. Dataset Source

Dataset used: Mall Customers Dataset

Source link:
https://www.kaggle.com/datasets/shwetabh123/mall-customers

2. Dataset Description

The dataset consists of customer information collected from a shopping mall. It is commonly used for segmentation analysis.

Features:

Feature	Description
CustomerID	Unique ID for each customer
Gender	Male/Female
Age	Age of the customer
Annual Income (k$)	Annual income in thousand dollars
Spending Score (1-100)	Score assigned based on spending behavior

3. Mathematical Formulation of the Algorithm

A) K-Means Clustering

K-Means partitions data into K clusters by minimizing the Within-Cluster Sum of Squares (WCSS):

$$J = \sum_{i=1}^{K} \sum_{x \in C_i} ||x - \mu_i||^2$$

B) Hierarchical Clustering (Ward Linkage)

Uses agglomerative approach (bottom-up).

Distance between clusters is calculated using Ward's method, minimizing variance:

$$D(A, B) = \frac{|A||B|}{|A| + |B|} ||\mu_A - \mu_B||^2$$

4. Algorithm Limitations

K-Means Limitations

- Requires predefined number of clusters (K)
- Sensitive to outliers
- Assumes clusters are spherical
- Results vary with initialization

Hierarchical Clustering Limitations

- High computational complexity
- Cannot undo cluster merges
- Sensitive to noise
- Difficult to scale for large datasets

5. Methodology / Workflow

Step-by-step process:

1. Load dataset (Mall_Customers.csv)
2. Select relevant numerical features:
    - Annual Income
    - Spending Score
3. Apply feature scaling using StandardScaler
4. Perform K-Means clustering (K = 3)
5. Assign cluster labels based on spending behavior
6. Compute Silhouette Score

7. Visualize clusters using scatter plot
8. Apply Hierarchical clustering using Ward linkage
9. Plot dendrogram to analyze cluster formation
10. Assign clusters using fcluster()
11. Visualize hierarchical clusters

6. Performance Analysis

Since clustering is unsupervised, evaluation is done using:

Silhouette Score

$$S = \frac{b - a}{\max(a, b)}$$

Interpretation:

- Score close to **1** → well-separated clusters
- Score near **0** → overlapping clusters

7. Hyperparameter Tuning

K-Means Tuning

Parameter: K (number of clusters)

Methods used:

- Elbow Method
- Silhouette Score comparison

Observation:

- K = 3 gave optimal clustering
- Balanced separation and interpretability

Hierarchical Clustering Tuning

Parameter: Linkage Method

Options tested:

- Single
- Complete
- Average
- Ward

Final choice:

- Ward linkage
    - Produces compact clusters
    - Minimizes variance

Source Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

from scipy.cluster.hierarchy import dendrogram, linkage, fcluster

# Load Dataset
df = pd.read_csv("sample_data/Mall_Customers.csv")

print("Total records:", len(df))
print(df.head())

# Feature Selection
# Using Annual Income & Spending Score
X = df[["Annual Income (k$)", "Spending Score (1-100)"]]

# Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

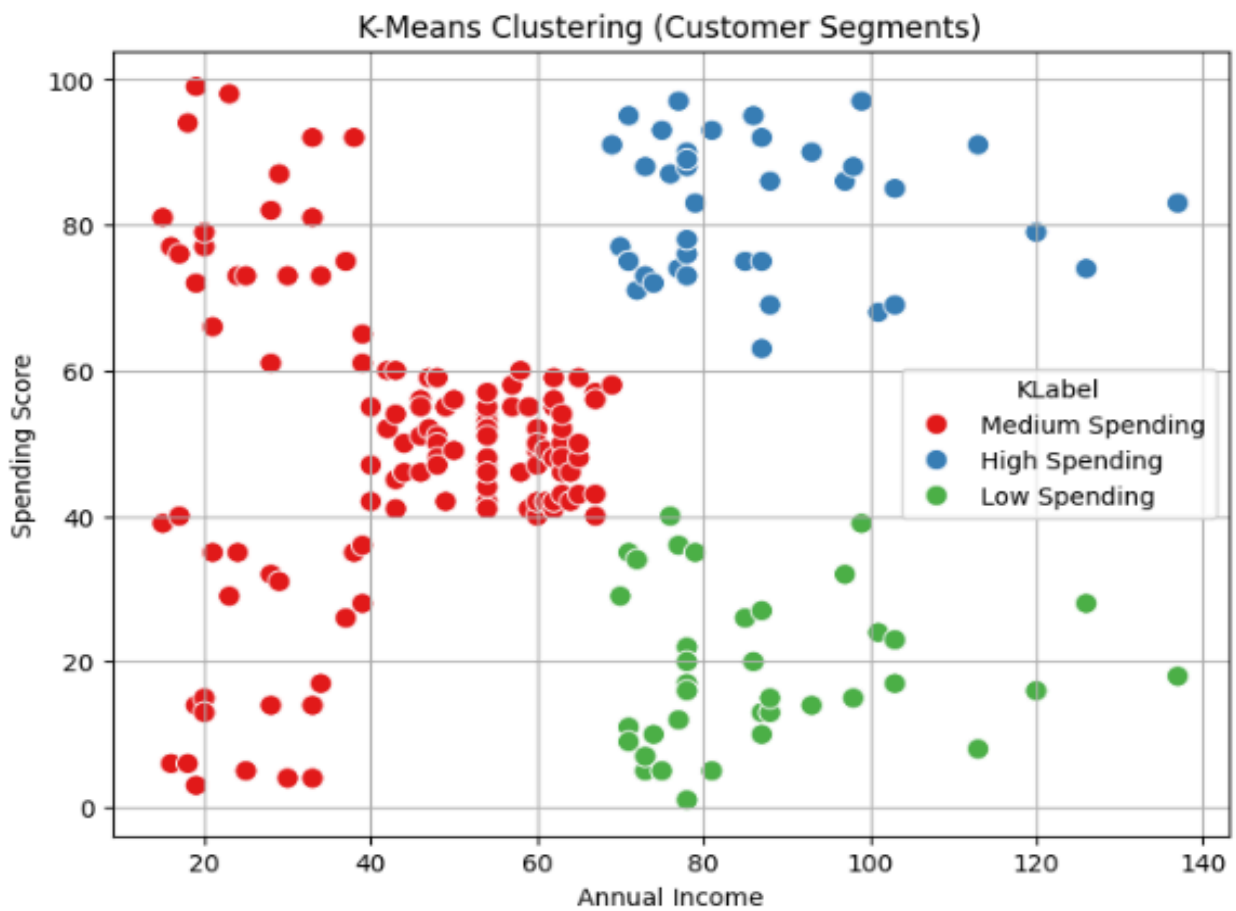
```
# K-MEANS CLUSTERING
kmeans_model = KMeans(n_clusters=3, random_state=10, n_init=10)
cluster_ids = kmeans_model.fit_predict(X_scaled)

df["KCluster"] = cluster_ids

# Label clusters (Low → High)
cluster_means = df.groupby("KCluster")["Spending Score (1-100)"].mean()
sorted_clusters = cluster_means.sort_values().index.tolist()

cluster_names = {
    sorted_clusters[0]: "Low Spending",
    sorted_clusters[1]: "Medium Spending",
    sorted_clusters[2]: "High Spending"
}

df["KLabel"] = df["KCluster"].map(cluster_names)

# Silhouette Score
score = silhouette_score(X_scaled, cluster_ids)
print("Silhouette Score:", score)

# K-MEANS VISUALIZATION
plt.figure(figsize=(8,6))

sns.scatterplot(
    x=df["Annual Income (k$)"],
    y=df["Spending Score (1-100)"],
    hue=df["KLabel"],
    palette="Set1",
    s=80
)

plt.title("K-Means Clustering (Customer Segments)")
plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.grid(True)
plt.show()

# HIERARCHICAL CLUSTERING
```

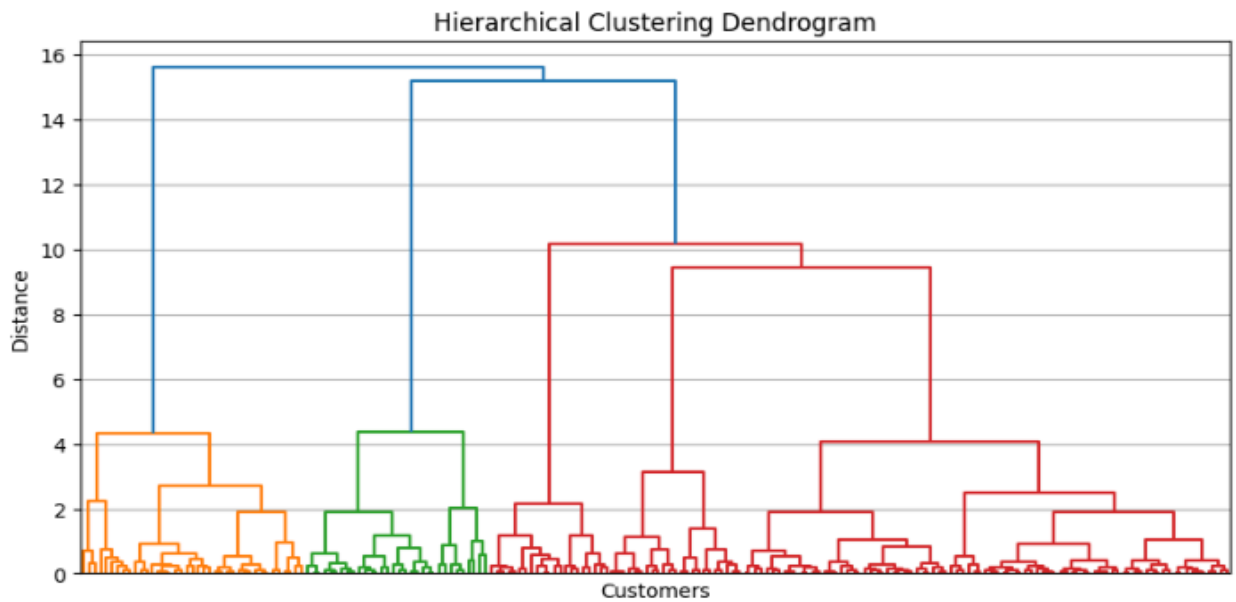
```
link_matrix = linkage(X_scaled, method="ward")

# Dendrogram
plt.figure(figsize=(10,5))
dendrogram(link_matrix, no_labels=True)
plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Customers")
plt.ylabel("Distance")
plt.grid(True)
plt.show()

# Create clusters
h_clusters = fcluster(link_matrix, 3, criterion="maxclust")
df["HCluster"] = h_clusters

# Label clusters
h_means = df.groupby("HCluster")["Spending Score (1-100)"].mean()
sorted_h = h_means.sort_values().index.tolist()

h_names = {
    sorted_h[0]: "Low Spending",
    sorted_h[1]: "Medium Spending",
    sorted_h[2]: "High Spending"
}

df["HLabel"] = df["HCluster"].map(h_names)

# HIERARCHICAL VISUALIZATION
plt.figure(figsize=(8,6))

sns.scatterplot(
    x=df["Annual Income (k$)"],
    y=df["Spending Score (1-100)"],
    hue=df["HLabel"],
    palette="Set2",
    s=80
)

plt.title("Hierarchical Clustering (Customer Segments)")
plt.xlabel("Annual Income")
```

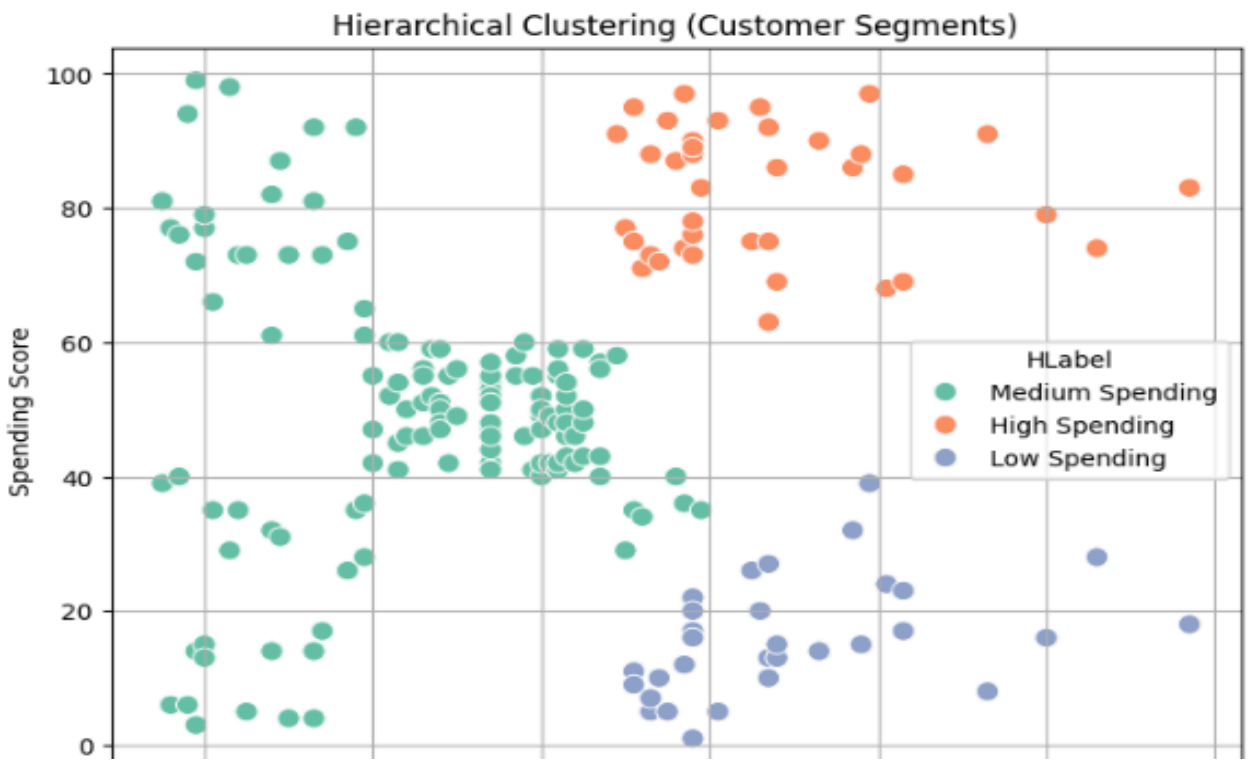
```
plt.ylabel("Spending Score")
plt.grid(True)
plt.show()
```

Output:

```
...  Total records: 200
        CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
     0           1    Male   19                  15                      39
     1           2    Male   21                  15                      81
     2           3  Female   20                  16                       6
     3           4  Female   23                  16                      77
     4           5  Female   31                  17                      40
     Silhouette Score: 0.46658474419000145
```







Conclusion:

Both K-Means and Hierarchical clustering successfully segmented customers into meaningful groups based on income and spending patterns. K-Means was computationally efficient and provided clear cluster boundaries, while Hierarchical clustering offered deeper insights through dendrogram visualization.